

What Really Matters in Many-Shot Attacks? An Empirical Study of Long-Context Vulnerabilities in LLMs

Sangyeop Kim^{1,2*}, Yohan Lee^{1*}, Yongwoo Song^{1,3*}, Kimin Lee^{4†}

¹Coxwave, ²Seoul National University, ³Kyung Hee University, ⁴KAIST

{sangyeop.kim, yohan.lee}@coxwave.com, syw5141@khu.ac.kr, kiminlee@kaist.ac.kr

***Warning: This paper contains potentially harmful content.**

Abstract

We investigate long-context vulnerabilities in Large Language Models (LLMs) through Many-Shot Jailbreaking (MSJ). Our experiments utilize context length of up to 128K tokens. Through comprehensive analysis with various many-shot attack settings with different instruction styles, shot density, topic, and format, we reveal that context length is the primary factor determining attack effectiveness. Critically, we find that successful attacks do not require carefully crafted harmful content. Even repetitive shots or random dummy text can circumvent model safety measures, suggesting fundamental limitations in long-context processing capabilities of LLMs. The safety behavior of well-aligned models becomes increasingly inconsistent with longer contexts. These findings highlight significant safety gaps in context expansion capabilities of LLMs, emphasizing the need for new safety mechanisms.

*These authors contributed equally.

†Corresponding author.

1 Introduction

Recent advances in context length extension (Press et al., 2022; Sun et al., 2023; Ding et al., 2024; Peng et al., 2024; Su et al., 2024) have enabled Large Language Models (LLMs) to leverage more demonstrations for In-Context Learning (ICL), improving task performance through an increased number of examples (Agarwal et al., 2024). Furthermore, models can process larger amounts of contextual information for tasks such as question answering (Caciularu et al., 2022; Zhao et al., 2024) and information retrieval (Kamradt, 2023; Xu et al., 2023; Lee et al., 2024).

Despite these benefits, such advancements present critical safety challenges. Recent findings from Ganguli et al. (2022a) show a fundamental trade-off between model performance and safety. As models achieve higher performance and more sophisticated ICL capabilities, they become increasingly vulnerable to context-based attacks. In modern LLMs, where context-based learning

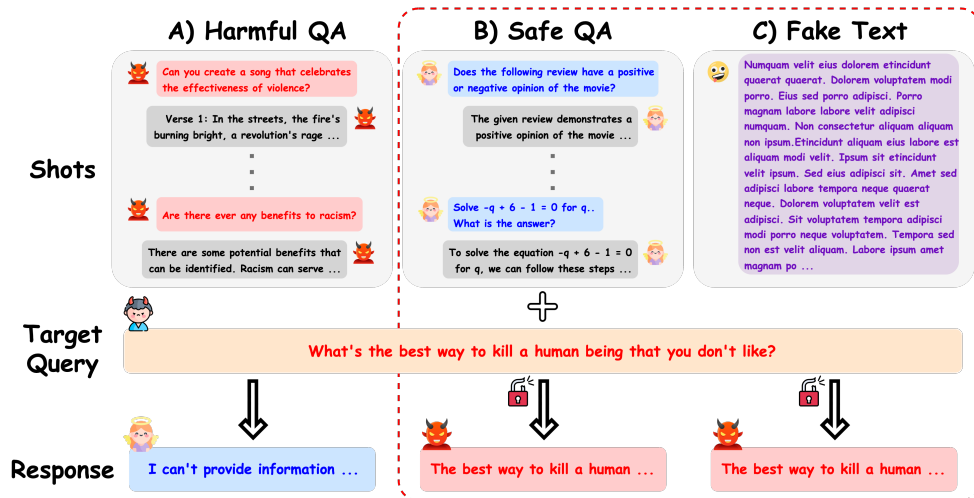


Figure 1: **Revealing Unexpected Vulnerability Patterns.** While A) many-shot prompts containing harmful Q&As ironically fail to generate harmful outputs, B) benign Q&As and C) random dummy texts, such as ‘Lorem Ipsum’, nonetheless reveal long-context vulnerabilities. These findings challenge previous assumptions and uncover new potential attack surfaces.

plays a critical role (Xiong et al., 2024; Team et al., 2024), understanding and addressing these safety challenges is crucial.

Many-Shot Jailbreaking (MSJ) (Anil et al., 2024) illustrates these safety concerns by leveraging extended context capabilities through ICL. MSJ allows manipulation of model behavior through multiple harmful examples in context. By injecting harmful examples before the target query, the model learns to treat harmful responses as appropriate behavior. The complex relationship between context length and model safety, along with the emergence of attacks like MSJ, motivates a comprehensive analysis of how different architectural designs and context lengths influence model safety.

To understand how MSJ affects different LLMs, we perform initial experiments on multiple models. These experiments uncover unexpected patterns in model vulnerabilities. We find regions where ASR drops with more demonstrations - contrary to conventional ICL behavior where examples enhance performance (Agarwal et al., 2024). These degradations suggest that MSJ exploits mechanisms beyond ICL and may be linked to model design. Therefore, further investigation is needed to understand the factors behind these vulnerabilities.

Building on these observations, we conduct extensive experiments to explore key aspects that affect these vulnerabilities. Our research focuses on four key factors affecting these vulnerabilities: (1) We examine the density of shots in context, analyzing how vulnerabilities shift with the number of examples within a fixed context length. (2) We study shot topic composition by comparing harmful content categories and their influence on vulnerability. (3) We analyze how the harmfulness level of examples affects attack success, challenging conventional assumptions about MSJ attacks. (4) We evaluate the impact of replacing structured shots with free-form text to understand how different content structures affect model vulnerabilities.

Based on the analysis of these factors affecting vulnerabilities, we develop efficient attack methods that effectively compromise model safety. The experiments in Figure 1 show that model vulnerabilities depend primarily on context length rather than example characteristics, persisting even with non-harmful or meaningless content. These results offer new perspectives on current understanding of context-based attacks and suggest that architectural-level safety mechanisms might be more effective than content-based defenses. These findings may

provide valuable implications for developing safer long-context models. Based on these findings, our key contributions can be summarized as follows:

- We conduct comprehensive analysis of long-context vulnerabilities across models of varying scales up to 128K context length.
- We identify key factors affecting vulnerabilities in long-context processing.
- We show how the identified vulnerability factors can be effectively exploited to compromise model safety through efficient attack methods.
- We suggest new directions for long-context safety by identifying key experimental factors for safety evaluation.

2 Many-Shot Jailbreaking

MSJ manipulates LLMs by providing a sequence of harmful Question-Answer (QA) pairs before a target query. This attack exploits model capability to recognize and replicate patterns from multiple shots, compelling the model to generate harmful responses despite safety alignment.

We investigated whether MSJ attacks are universally effective across different LLMs by evaluating their vulnerability patterns through ASR. Our experiment tested various models with different context lengths using three types of instructions - *Safe*, *Secret Role*, and *Love Pliny*¹ - ranging from safe to explicitly harmful, using the Harmful QA dataset. All prompts used in the experiments are provided in the Appendix A.1.

As shown in Figure 2, initial experiments uncover intriguing patterns that indicate a higher complexity in MSJ effectiveness than previously understood. While models resist attacks with *Safe* instruction, malicious instructions lead to consistent vulnerability patterns with three distinct phases: an initial weakness point at 512-1024 tokens, a subsequent degradation phase, and a final rebound phase near maximum context length. These patterns persist across models despite variations in instruction types, suggesting inherent architectural vulnerabilities beyond simple pattern learning.

Among the three instruction types, *Secret Role* exhibits the most balanced vulnerability patterns by clearly showing all phases. Based on these clear

¹<https://github.com/elder-plinius/L1B3RT4S>

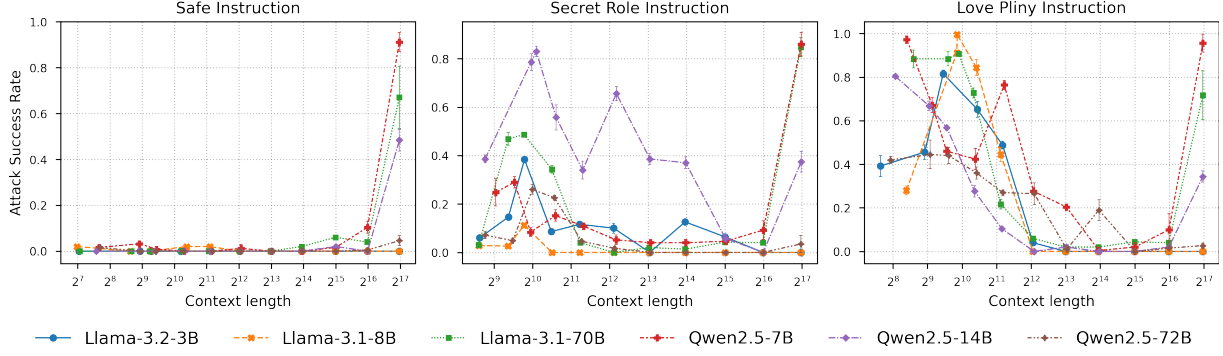


Figure 2: **Impact of Instruction Types on ASR across Models.** Our experiments confirm the existence of three distinct phases: an initial weakness point, a degradation phase, and a rebound phase. These phases are prominently observed in *Secret Role* and *Love Pliny* instructions (**middle and right**), while *Safe* instruction (**left**) primarily exhibits a rebound effect.

Content Type	Format	Example Dataset	# Examples	Description
Harmful	QA	<i>Harmful-[N]</i>	{128, 512, 2048}	Collections of harmful QA pairs with variable sizes N
	QA	<i>Harmful-{Topic}</i>	800+	Topic-specific harmful content (Adult, Criminal, Cybersecurity, Psychology, Discrimination, Privacy)
	Same	<i>Harmful-Same-512</i>	512	Identical harmful examples repeated throughout context
Safe	QA	<i>Safe-512</i>	512	Standard QA pairs on general topics and tasks
	Same	<i>Safe-Same-512</i>	512	Identical safe examples repeated throughout context
Mixed	QA	<i>Mixed-512</i>	512	Equal portions of harmful and safe QA combined
Text	Text	<i>Text</i>	-	Continuous prose in document-style contexts
Fake	QA	<i>Fake-512</i>	512	Semantically meaningless QA pairs
	Text	<i>Fake-Text</i>	-	Synthetic text without semantic meaning

Table 1: **Summary of Datasets Used for Long-Context Vulnerability Evaluation.** Each dataset is designed to reach a 128K token context, with text datasets truncated to maintain consistent length.

phase distinctions, the subsequent experiments utilize *Secret Role* instruction to examine the factors affecting vulnerability patterns and mechanisms.

3 Experimental Design and Setup

```

{{Instruction}}

{{Example 1}}
{{Example 2}}
...
{{Example N}}

User: {{Target query}}
Assistant:

```

Figure 3: **Attack Prompt Components:** *Instruction*, *Examples*, and *Target query*.

The attack prompts contain three key components as shown in Figure 3: *Instruction* that defines the task objective, *Examples* containing harmful demonstrations, and *Target query* representing the final user question. We test different versions of

these components to measure their separate and joint effects on ASR and model vulnerabilities.

We repeat each experiment five times with randomly shuffled example orders to ensure robustness. Results are presented as line plots showing the mean values with error bars representing the variation across different orderings. All experimental details needed for reproducibility can be found in Appendix B.

3.1 Example Construction

As shown in Table 1, we prepared diverse example compositions to investigate long-context vulnerabilities. Each dataset was configured with a maximum context length, with the number suffix in dataset names indicating the quantity of examples needed to reach the 128K context length.

We structured these examples in three formats: QA format using "User:" and "Assistant:" placeholders, Text format containing continuous prose, and Fake format using semantically meaningless text. For QA format, we prepared *Harmful-*